



1. Resumen

Se publicó la misma telemetría (Temperature, Humidity, Illuminance; JSON plano de 60 B) desde una VM propia hacia Azure IoT Central con tres protocolos: **MQTT** (línea base del Lab 3), **AMQP 1.0** y **CoAP** sobre UDP como tercer protocolo (con gateway propio, porque no es nativo de la plataforma). Cada protocolo usa un device dedicado en la app *clima-salones-app-jose* (*mqtt-baseline-01*, *amqp-lab4-01*, *coap-gateway-01*) para no mezclar telemetría. **Conclusión:** en estado estacionario la latencia de los tres es prácticamente la misma (110.8 / 111.8 / 118.9 ms), porque la domina la ruta hacia Azure y no el protocolo; lo que cambia es el costo de arranque (AMQP), la arquitectura (CoAP exige gateway) y la complejidad de implementación. Recomendación: **MQTT en el dispositivo y AMQP entre servicios**. Este informe usa referencias (**Evid. N**) a las capturas reunidas en el **Anexo de evidencias** (págs. 3-4).

2. El reto antes del reto: conseguir una VM

Las VMs de los labs anteriores (57.156.66.112 y 57.156.62.111) no respondían en 22, 80, 443, 8883 ni 5671 (timeout) y se perdieron por una causa desconocida. El redespigüe en Mexico Central falló con RequestDisallowedByAzure (Azure Policy); Chile Central tampoco estaba autorizada. En lugar de probar regiones al azar, desde Cloud Shell se consultó `az policy assignment list --query "[].parameters":` la política solo permite *centralus*, *eastus*, *westus3*, *northcentralus* y *brazilsouth*. Cruzado con el tamaño Standard_B2ats_v2, solo **North Central US** cumplía: VM *vmtallerjose* (Ubuntu 24.04, 52.237.172.24). Además, la app de IoT Central había sido recreada (ID Scope pasó de One0128547D a One0128155B y la plantilla/devices del Lab 3 daban 404), por lo que *provision_devices.py* recrea la plantilla *consola-unab-ambiental* y los tres devices por REST API (hallazgo en *evidencias/vm_azure_policy_hallazgo.md*).

3. Rol de AMQP en Azure frente a MQTT, y elección de CoAP

AMQP 1.0 es el protocolo del plano de servicio de Azure (Service Bus, Event Hubs) y un transporte nativo de IoT Hub / IoT Central (5671, o 443 con AMQP sobre WebSockets). Frente al pub/sub ligero de MQTT aporta **sesiones, links con control de flujo por créditos y disposición de entrega explícita por mensaje** (accepted / rejected / released), útil entre servicios que mantienen el link abierto; MQTT es más simple y mejor soportado en microcontroladores. **Hallazgo:** *azure-iot-device* 2.x (2.14.0 y 2.13.0 auditados) ya no incluye AMQP: `transport="Amqp"` da *TypeError* y el paquete solo trae pipelines MQTT y HTTP. Se escribió entonces un cliente AMQP explícito sobre *uamqp* 1.6.11: SAS token firmado en memoria (HMAC-SHA256), autenticación CBS (*put-token a \$cbs*) y link a `amqps://{hub}/devices/{id}/messages/events`. **CoAP (RFC 7252)** se eligió por contraste (UDP, cabecera mínima, pensado para dispositivos constrained). Justificación de negocio: en un campus con sensores ambientales de muy bajo consumo, el nodo no debería sostener TLS ni una conexión persistente; un gateway en el borde puede hacerlo por él.

4. Arquitectura implementada (Evid. 1)

Los tres clientes corren en la misma VM. MQTT (TCP+TLS, 8883) y AMQP (TCP+TLS, 5671) llegan directo al hub de IoT Central asignado por DPS; **CoAP no:** *coap_client.py* (aiocoap) hace POST CON `/telemetry` por UDP 5683 a *coap_gateway.py* (escucha en 127.0.0.1, sin TLS en ese salto, DTLS no usado), que valida el JSON y lo reenvía por MQTT/TLS con el device *coap-gateway-01*; responde *2.04 Changed* después de reenviar, por lo que su RTT incluye el tramo hacia Azure. Todo comparte *common.py* (variables con caminata aleatoria ± 0.5 en rangos DTDL, payload de 60 B exactos, DPS idempotente con reintentos y caché, firma del SAS), así que lo único que cambia es el protocolo.

Elemento	MQTT	AMQP 1.0	CoAP
Cliente / device	<i>mqtt_baseline.py</i> (paho 1.6) · <i>mqtt-baseline-01</i>	<i>amqp_sdk.py</i> (uamqp) · <i>amqp-lab4-01</i>	<i>coap_client.py</i> + <i>coap_gateway.py</i> · <i>coap-gateway-01</i>
Puerto · transporte · seguridad	8883 · TCP · TLS 1.2/1.3 + SAS en CONNECT	5671 · TCP · TLS + CBS	5683/udp · sin TLS en el salto CoAP; 8883 TLS en el puente
Mensaje · evidencia	PUBLISH QoS 1 a <code>devices/{id}/messages/events/</code> · PUBACK en cada uno	transfer al link events · 9/9 SendComplete	POST CON <code>/telemetry</code> · 8/8 respuestas 2.04 Changed
Complejidad real	Baja: paho + fórmula SAS del Lab 3	Alta: SDK sin AMQP → uamqp + CBS manual + link propio	Media: aiocoap sencillo, pero hay que construir y operar el gateway
Problema → solución	DPS 401 (.env con ID de device eliminado) → corregir MQTT_DEVICE_ID	<i>TypeError</i> de transport; DPS limitando ráfagas → cliente uamqp explícito, backoff y caché	<code>add_resource/bind</code> 0.0.0.0; moría al cerrar SSH; identidad duplicada → Site, bind 127.0.0.1, setsid, matar proceso duplicado

5. Evidencia de funcionamiento y mediciones (Evid. 2 a 7)

Los tres devices aparecen aprovisionados y no simulados en IoT Central (**Evid. 2**); en el Data explorer (grupo *consola-unab-ambiental* - *All devices*, intervalo 20 s) se ven **Humidity, Illuminance y Temperature de los tres devices fluyendo a la vez**, lo que prueba que AMQP y CoAP (vía gateway) entregan lo mismo que MQTT (**Evid. 3**). Las latencias miden el tiempo hasta la confirmación (PUBACK en MQTT; *disposition accepted* en AMQP; *2.04 Changed* en CoAP, que incluye el reenvío a Central): no son idénticas, pero sí comparables en espíritu. El payload es idéntico (60 B) en los tres (**Evid. 7**).

Protocolo · puerto	Corrida formal (90 s, 1 msg/10 s)	Captura larga (3 s): n · prom · mín-máx	Cabecera	Observación
MQTT · 8883	n=9 · 114.9 ms · 104.0-124.3	392 · 110.8 ms · 102.9-169.1	2 B	PUBACK QoS 1
AMQP 1.0 · 5671	n=9 · 148.9 ms · 100.1-430.1 (sin 1.er msg ≈114.5)	392 · 112.7 ms · 101.0-461.6 (estacionario 111.8)	8 B	1.er mensaje = handshake TLS+CBS+attach
CoAP · 5683/udp	n=8 · 120.9 ms RTT · 110.7-127.6 (gateway→Central 119.0)	394 · 118.9 ms RTT · 104.2-166.1 (forwarding 117.4)	4 B	Salto UDP local ≈ 1.5 ms

Las gráficas de la captura (**Evid. 4, 5, 6**) muestran los promedios, la distribución (sin el 1.er mensaje AMQP) y la latencia por muestra: pico inicial de AMQP (≈460 ms) = handshake; picos aislados en AMQP (≈400 ms hacia la muestra 200, ≈200 ms hacia la 35) y MQTT (≈170 ms) cuya causa no se investigó. **Local vs VM:** desde el portátil (fuera de Azure) las mismas pruebas dieron 176.4 ms (MQTT, n=4), 175.0 (AMQP) y 174.5 (CoAP), y el primer mensaje AMQP tardó 1433 ms; desde la VM MQTT bajó a 114.9 ms (≈ 35 % menos solo por proximidad de red). Antes de comparar protocolos hay que comparar *desde dónde* se mide.

6. Tabla comparativa: observado (O) vs teórico (T)

Criterio	MQTT	AMQP 1.0	CoAP (UDP)	Tipo
Establecimiento	TCP+TLS+CONNECT con SAS → CONNACK	TCP+TLS → CBS put-token → attach → transfer (1.º: 430-460 ms)	Sin conexión: cada POST es un datagrama	O
Modelo	Pub/sub por topics	Peer-to-peer con links y créditos	Request/response (CON)	T
Overhead	Cabecera 2 B; el costo real está en topic y tramas TCP/TLS	Trama 8 B; el costo real es el handshake	Cabecera 4 B; sin handshake	O+T
Latencia percibida	110.8 ms	112.7 (estac. 111.8; 1.º 461.6)	118.9 RTT (fwd 117.4)	O
Fiabilidad	PUBACK QoS 1: 100 %	SendComplete: 100 %	2.04 Changed 8/8; retransmisión CON teórica (no se provocó pérdida)	O+T
Recursos CPU/RAM	Conexión persistente, ligero	Sesión + link + TLS persistentes	Sin estado de conexión en el nodo	T
Seguridad	TLS + SAS token	TLS + CBS con SAS	Sin TLS en salto CoAP; gateway en loopback; TLS en el puente	O
Firewall / NAT	8883 a veces bloqueado (alt. 443 WebSockets)	5671 (alt. 443 AMQP-WS)	UDP 5683 suele pasar, sin garantías	T
MCU / Wokwi	Maduro (paho/Arduino/ESP32)	Poco usado en MCU	Ideal para constrained (RFC 7252)	T
Integración Central · complejidad	Nativa · baja · debug muy fácil	Nativa · alta (SDK sin AMQP) · debug medio (tramas binarias)	Requiere gateway propio · media · debug medio-fácil	O
Caso de uso ideal	Telemetría ligera de muchos sensores / MCU	Mensajería entre servicios Azure	Nodo constrained en red local con gateway	T

Lo observado: (1) la latencia estacionaria no distingue a MQTT y AMQP (≈ 1 ms de diferencia; CoAP ≈ 8 ms por el gateway); (2) AMQP se paga al arrancar: el primer mensaje costó $\approx 4\times$ el promedio, un costo que se repetiría en un sensor que despierta, publica y duerme y que se amortiza en un servicio con el link abierto; (3) CoAP simplifica el nodo pero complica la arquitectura, y el gateway fue la fuente del único problema operativo serio; (4) “nativo” no significa “fácil”: AMQP es nativo y fue el más difícil de implementar. **Limitaciones:** no se midió CPU/RAM, no se inyectó pérdida de paquetes y no se probaron las variantes sobre WebSockets (443).

7. Sala de control didáctica (milla extra) (Evid. 8, 9)

En la VM se construyó una sala de control pública (<https://iotcentraljose.duckdns.org>; Flask + gunicorn en systemd, nginx 443 con Let's Encrypt, puerto 8080 cerrado) para *ver viajar* los protocolos: tarjeta por protocolo con contador, latencia última/promedio/mín/máx, payload actual, sparkline y salida cruda; sección *Cómo viaja el mensaje* con carrera de latencia, pila de protocolos, peso del mensaje y secuencia real del último intercambio (**Evid. 8**); glosario de 12 términos y comparativa en vivo (**Evid. 9**). Inicia/detiene los clientes desde el navegador (acciones protegidas con la clave DASH_KEY, solo en el .env de la VM) y una corrida cronometrada de 90 s captura sola la evidencia: *.jsonl, resumen.json, reporte.md, cuatro gráficas PNG, página imprimible /informe y .zip (36 archivos).

8. Otros problemas relevantes

Gateway CoAP en bucle de ConnectionDroppedError: no era del código ni del SDK; lanzado con nohup ... & desde SSH, el proceso moría con SIGHUP al cerrar la sesión. Aislar la variable (primer plano vs segundo plano) lo reveló; se lanzó con setsid nohup ... < /dev/null. **“Unexpected disconnection”:** dos procesos con la misma identidad (uno local seguía vivo) se desconectaban mutuamente. En la sala de control: `kill -f` desde SSH se mataba a sí mismo (se paró por PID), la petición HTTP se colgaba por un pipe heredado (Popen con DEVNULL y `start_new_session`), matplotlib 3.11 renombró `labels` a `tick_labels`, y el 1.er mensaje AMQP aplastaba la escala del boxplot (se excluyó y se anotó).

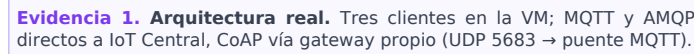
9. Recomendaciones para la plataforma propia

- **Dispositivo (Wokwi / ESP32 futuro): MQTT.** Soporte maduro en MCU, QoS 1, debug muy fácil, latencia equivalente a AMQP (110.8 vs 111.8 ms) y la implementación más simple.
- **Entre servicios Azure / plataforma propia (Labs 5-8): AMQP.** Sesiones, créditos y disposición por mensaje; el handshake de ≈ 430 -460 ms se amortiza porque el servicio mantiene el link abierto.
- **Quedarse con dos protocolos: MQTT + AMQP.** Cubren dispositivo → nube y servicio → servicio, ambos nativos de IoT Central. CoAP + gateway solo si un nodo constrained no puede sostener TLS/TCP, asumiendo operar el gateway.
- **Prácticas:** todo proceso persistente como servicio (systemd); una identidad, un proceso; medir desde donde correrá; reportar aparte el primer mensaje de AMQP; no fingir integración nativa (el gateway se muestra en el diagrama); credenciales fuera del repo (variables de entorno, SAS en memoria, .env en .gitignore).

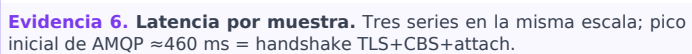
10. Repositorio y cómo reproducir

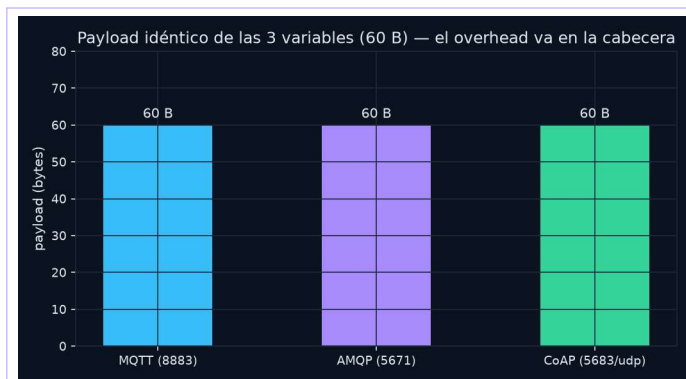
Taller 4 en github.com/joserraldolot_learning: `python/` (common, mqtt_baseline, amqp_sdk, coap_client, coap_gateway, provision_devices), `dashboard/` (sala de control), `tools/` (despliegue), `README.md` (dependencias y pasos: `cp .env.example .env`; sin secretos) y `/evidencias` (mqtt, amqp, coap, comparativa, capturas). Corridas formales en la VM: `.venv/bin/python -u python/mqtt_baseline.py --duration 90 --interval 10`, igual con `amqp_sdk.py`; para CoAP, lanzar `coap_gateway.py` con `setsid` y luego `coap_client.py`; o bien «Iniciar los 3» en la sala de control. **Uso de IA generativa:** se empleó Claude (Anthropic) para estructurar y redactar este informe a partir de la bitácora, los logs y las capturas reales; las mediciones provienen de las corridas del autor y el contenido fue revisado por él.

Capturas y gráficas reales del laboratorio (Informe extendido, 15 págs.), numeradas según las referencias (**Evid. N**) del informe. Archivos originales en /evidencias del repositorio.

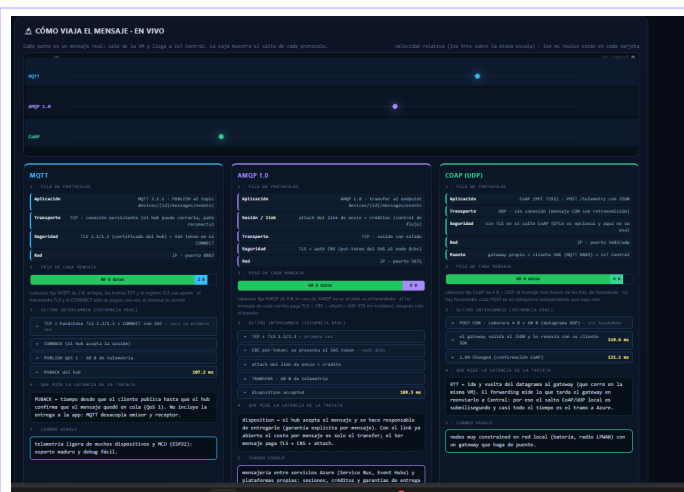


Evidencia 2. IoT Central · Devices. amqp-lab4-01, coap-gateway-01 y mqtt-baseline-01: Provisioned, no simulados (uno por protocolo).





Evidencia 7. Payload idéntico de 60 B. El overhead no está en los datos, está en la cabecera y el handshake.



Evidencia 8. Sala de control «Cómo viaja el mensaje». Carrera de latencia, pila de protocolos, peso del mensaje y secuencia real del último intercambio.



Evidencia 9. Sala de control · glosario y comparativa en vivo. Barras y serie temporal; indicador «3/3 protocolos en vivo» y candado HTTPS.

Índice de evidencias

N.º	Qué demuestra	Se referencia en
1	Arquitectura real y ruta de cada protocolo (CoAP vía gateway)	Sec. 4
2	Los 3 devices dedicados aprovisionados en IoT Central	Sec. 5
3	Las 3 variables de los 3 protocolos llegando a IoT Central (AMQP incluido)	Sec. 5 · checkpoint AMQP
4-6	Latencia promedio, distribución y por muestra (captura larga en la VM)	Secs. 5 y 6
7	Payload idéntico de 60 B en los tres protocolos	Sec. 5
8-9	Sala de control: cómo viaja el mensaje, glosario y comparativa en vivo; HTTPS	Sec. 7